



Home



My Network



Jobs



Messaging



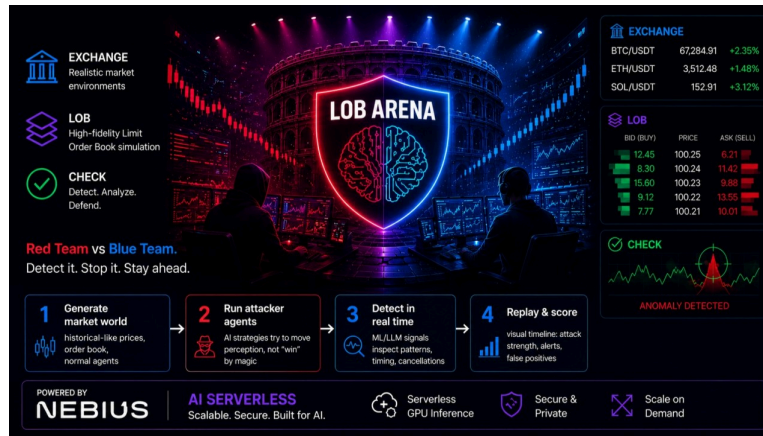
Notifications



Me



For Business

[Edit article](#)[View stats](#)[View post](#)

Arena is a synthetic arena for evaluating market-abuse detectors and AI-assisted investigations on Nebius Serverless AI.

Building LOB Arena: An Adversarial Market-Surveillance Evaluation Platform with Nebius Serverless AI

**Alexey Khabalov** ✓

Engineering Director / Senior Manager | Data Platforms, Market Data, FinTech, AI | EU...



July 15, 2026

Market-surveillance teams can purchase historical order-book data from exchanges and specialist market-data vendors. The harder problem is obtaining complete, reliable ground-truth labels that identify which sequences represent manipulation, which reflect legitimate trading behaviour, and how a surveillance system should respond.

Even with high-quality historical data, benchmarking remains difficult. Teams need repeatable scenarios, known agent intentions, consistent ground truth, controlled market conditions, and comparable metrics across detector designs and market regimes.

This was the motivation for **LOB Arena**.

LOB Arena is an open-source platform for generating synthetic limit-order-book activity, injecting bounded abuse-like scenarios, benchmarking surveillance detectors, and using Nebius Serverless AI to produce structured investigations and explanations.

Repository: <https://github.com/khab40/lob-arena>

I built the project for the **#NebiusServerlessChallenge** as an engineering environment rather than as a claim that an AI model can independently detect real market manipulation.

The goal is to create a controlled, reproducible arena in which the ground truth is known because the scenarios are generated deliberately.



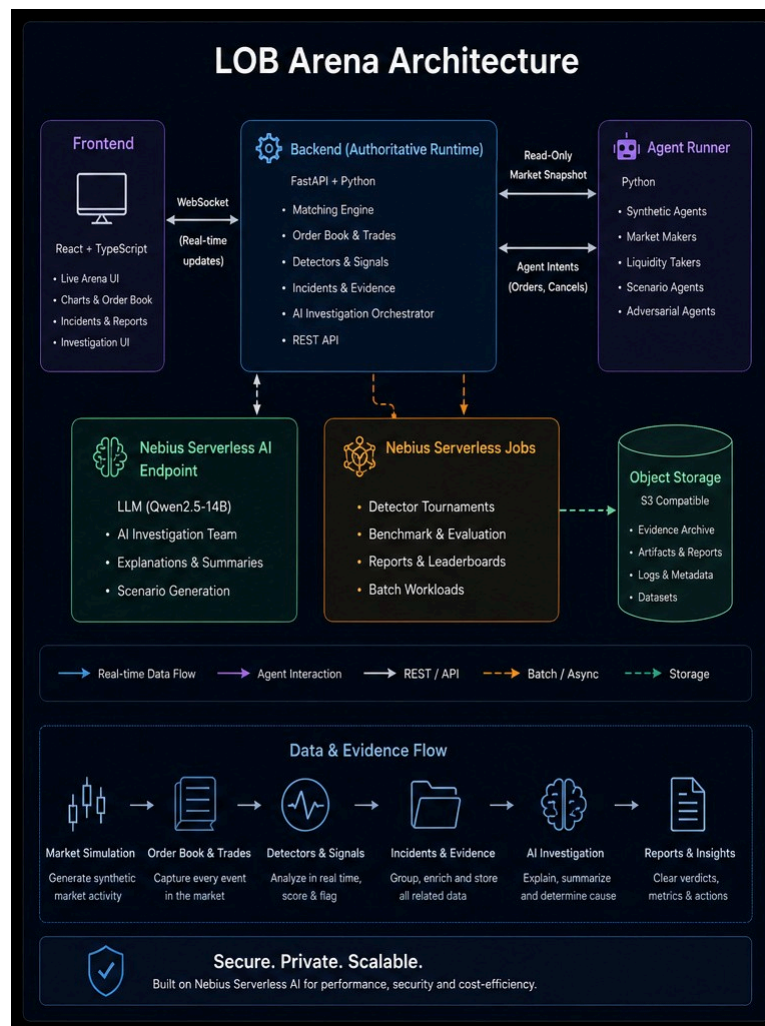
LOB Arena's functional flow: simulate bounded market-abuse scenarios, detect suspicious behaviour, create incidents, investigate with AI, and benchmark detector performance.

That creates a concrete engineering surface: generate market events, inject labeled scenarios, run deterministic

detectors, preserve structured evidence, and use AI to explain what the detector already found.

The architecture has two main execution paths.

The interactive path uses a React and Vite frontend, a FastAPI control plane, a separate agents workspace, and a Nebius Serverless AI Endpoint. The batch path uses Nebius Serverless AI Jobs for repeatable synthetic workloads, detector evaluation, aggregation, and artifact generation.



LOB Arena separates the React interface, authoritative FastAPI runtime, agents workspace, and Nebius Endpoint and Job execution paths.

LOB Arena was validated on real Nebius production infrastructure. More than ten Nebius Serverless AI Job runs completed successfully, with execution visible in production logs. I also deployed a vLLM-backed Nebius Serverless AI Endpoint and exercised routes for scenario generation, incident analysis, investigation reporting, order-book alert analysis, and structured market-event explanation. Those runs produced Job artifacts, detector metrics, reports, logs, and Endpoint responses. This

validation proves the execution contracts; it does not turn LOB Arena into a real-market surveillance product.

Job lifecycle records and generated artifacts are archived to Nebius Object Storage. Endpoint execution metadata is archived through the same evidence layer without presenting private credentials or sensitive transport details to the browser. The backend can synchronize archived evidence from S3-compatible storage back to backend-local storage, and the UI exposes the synchronized records and downloadable artifacts. This creates a traceable path from production execution, through durable storage, to evidence that a reviewer can inspect.

The interactive path

The frontend renders the live arena: order-book ladders, price and spread charts, liquidity heatmaps, agent activity, detector confidence, incident cards, replay, and report views.

The browser sends commands over WebSocket. The FastAPI backend runs the simulation and publishes complete `arena_state` messages. This keeps the browser away from simulation internals, server credentials, and direct Endpoint access.

During an interactive run, LOB Arena uses the separate agent-runner/ workspace to generate normal synthetic market activity.

At each simulation tick, the backend sends a read-only order-book snapshot to the workspace through its `/decide` API. The workspace returns typed `AgentIntent` objects rather than mutating the market directly. The backend validates and deterministically sorts those intents, remains the single authoritative writer, and applies accepted actions to the synthetic exchange and matching engine.

The workspace runs several kinds of trading agents. Top-of-book market makers refresh visible liquidity on both sides. Deterministic noise traders make small cadence-based changes at selected levels. Periodic liquidity takers alternate bounded synthetic buys and sells. Optional LangGraph agents can choose which side to quote from observed depth imbalance. Optional CPU-heavy agents exercise a more computationally expensive decision path for workload testing.

This separation lets the frontend display agent activity while the backend preserves ordering, timeouts, validation, and reproducibility.

None of these agents connects to a broker, exchange, or real market. They trade only inside LOB Arena's synthetic order book and cannot emit real orders or trading signals.

Inside the backend, the core loop is intentionally deterministic. A synthetic exchange, order book, and matching engine process actions from market-making, liquidity-taking, and noise agents. Scenario agents can then inject the four implemented workloads: Spoofing-like Wall, Layering-like Pattern, Quote Stuffing Burst, and Liquidity Evaporation.

The key word is "bounded." These are synthetic patterns for education and detector testing, not instructions for real market activity.

Why detection and explanation are separate

Nebius Serverless AI Endpoints do not make LOB Arena's original detection decision.

A deterministic detector produces structured evidence first: spread, visible depth, imbalance, message rate, cancel-to-trade ratio, wall-size ratio, order lifetime, confidence scores, and scenario labels.

The backend then sends a compact incident payload to the Endpoint. The Endpoint can return a readable explanation, investigation assistance, recommended review actions, or a bounded synthetic scenario draft.

This split matters because it keeps the workflow auditable. AI is used for explanation, narration, investigation assistance, and bounded scenario generation. Structured detector evidence remains the source of truth.

detector signals, heatmaps, and recent trades.

7. STRUCTURED JSON OUTPUT

Strict contract for investigations and explanations

```

1  {
2    "classification": "layering",
3    "severity": "high",
3    "confidence": 0.87,
4    "summary": "The trader placed and cancelled large
5      orders across multiple levels to create a false
6      sense of liquidity.",
7    "evidence": [
8      { "type": "large_cancellations", "value": 0.82 },
9      { "type": "wall_size_ratio", "value": 0.76 },
10     { "type": "spread_widening", "value": 0.71 }
11   ],
12   "counter_evidence": [
13     { "type": "some_executions", "value": 0.35 }
14   ],
15   "alternative_explanations": [
16     "legitimate_market_making", "inventory_rebalancing"
17   ],
18   "recommended_actions": [
19     "review_order_history", "check_trader_intent",
19     "escalate_if_persistent"
20   ],
21   "regulatory_framing": "Consistent with layering behavior
21     as described in MAR and CFTC guidelines.",
22   "detector_disagreement": [
23     { "detector": "liquidity_evaporation", "score": 0.42 }
24   ]
25 }

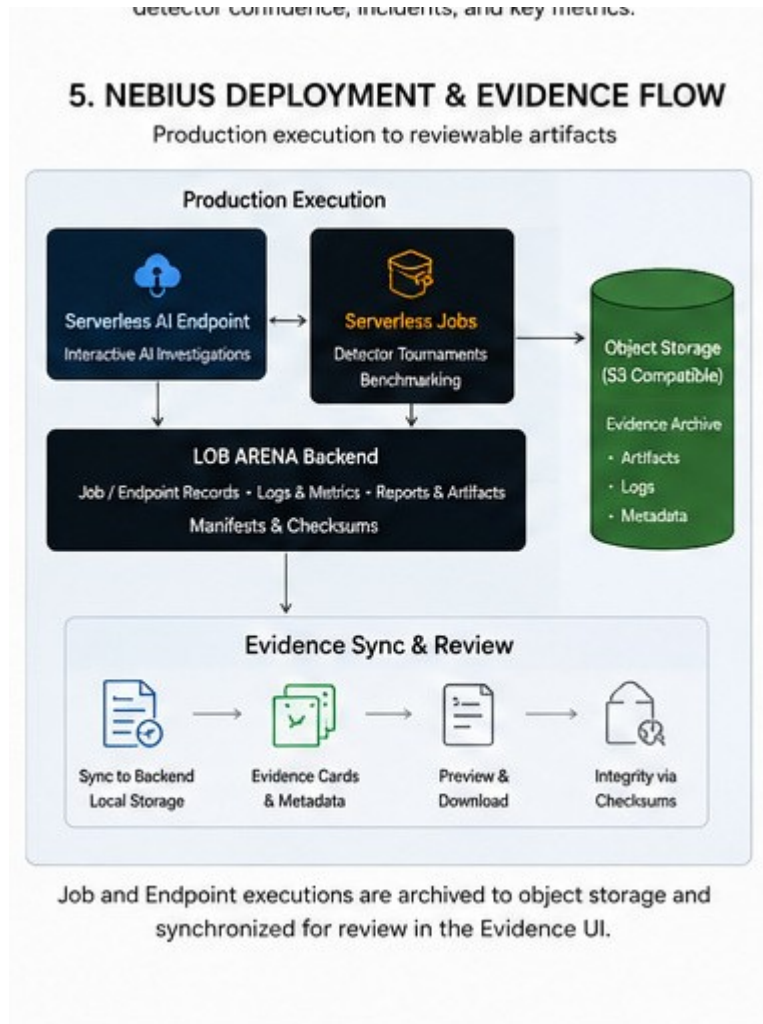
```

AI responses follow a strict JSON schema for reliable parsing and consistent presentation.

Endpoint responses are parsed as structured JSON so the UI can separate classification, confidence, evidence, counter-evidence, and recommended actions.

The batch path

Nebius Serverless AI Jobs fit the offline evaluation path naturally. Instead of asking a live request to run dozens or hundreds of simulations, a Job can execute repeatable synthetic workloads, evaluate detector output against labels, aggregate metrics, and persist reports and artifacts before terminating.



Production Job and Endpoint evidence is archived to Object Storage, synchronized by the backend, and exposed as reviewable UI records and downloads.

The outputs are designed to be reviewable: JSON records, CSV metrics, Markdown reports, logs, and chart-ready data. The metric vocabulary includes precision, recall, F1, false positives, false negatives, and detection latency against known synthetic labels.

The repository is organized around those boundaries.

backend/ contains the FastAPI application, simulation engine, exchange model, detectors, incident storage, Nebius client, evidence synchronization, and report generation.

agent-runner/ contains the agents workspace service, its /health, /agents, and /decide contracts, normal synthetic agents, optional CPU-heavy agents, and bounded LangGraph strategies.

frontend/ contains the React arena, Detection workflow, Scenario Generator, Nebius AI controls, evidence views,

and visualization components.

serverless/ contains the Endpoint application, Job runners, Dockerfiles, and example deployment configurations.

docs/ contains architecture decisions, deployment notes, benchmark methodology, safety framing, and challenge-submission material.

outputs/benchmark/ contains the public sanitized benchmark and production-evidence sample.

Four evidence layers

I use four explicit names in the repository so small integration checks are not confused with production validation.

Smoke Contract Test

The **Smoke Contract Test** is the smallest end-to-end integration check. It verifies that scenario generation, simulation, detector routing, investigation output, tournament execution, and artifact contracts connect correctly.

The deliberately small one-run detector example covers spoofing-like and quote-stuffing scenarios. Matching detectors reach precision 1.0, recall 1.0, and F1 1.0, with an average detection latency of 1,500 ms in that fixture.

Those perfect values are integration evidence only. They validate labels, routing, metric calculation, and artifact persistence on a deliberately simple deterministic fixture. They are not evidence of real-world surveillance accuracy, robustness, or compliance suitability.

Local Reproducibility Benchmark

The **Local Reproducibility Benchmark** is the deterministic path another practitioner can run with Docker Compose. It generates labeled synthetic workloads and produces metrics, leaderboards, reports, and artifacts without requiring Nebius credentials.

Its purpose is reproducibility: the same configuration and seed should preserve the evaluation contract and make failures inspectable. Local execution is not presented as proof that a cloud Job or Endpoint ran.

Production Execution Validation

The **Production Execution Validation** covers the real Nebius infrastructure path. More than ten Serverless AI Job runs validated container startup, scenario execution, detector evaluation, aggregation, reporting, logging, and artifact persistence. The vLLM-backed Endpoint separately validated interactive routes.

This layer answers a deployment question: do the packaged Job and Endpoint contracts operate on production infrastructure and produce reviewable outputs? It does not answer whether the synthetic detector generalizes to real markets.

Representative Production Run

The **Representative Production Run** is the compact, sanitized evidence sample committed for review. It preserves Job records, normalized detector metrics, a detector/model leaderboard, investigation results, an evidence index, a benchmark report, a manifest, and checksums.

The public bundle intentionally excludes credentials, authorization headers, private Endpoint hostnames, signed URLs, and raw environment-specific logs. It also reconciles the benchmark denominator explicitly: the representative 100-workload run contains 80 labeled attack rows and 20 normal-market control rows. A judge can inspect the evidence without access to the private Nebius account or Object Storage bucket.



Detector Tournament results connect labeled synthetic workloads to metrics, leaderboards, reports, and sanitized production-execution evidence.

Why Nebius Serverless AI

LOB Arena uses Nebius Serverless AI in two materially different ways.

The **Serverless AI Endpoint** supports interactive operations such as AI Investigation Team analysis, incident explanation, structured investigation reports, bounded synthetic scenario generation, and market-event summarization. The representative deployment uses an NVIDIA L40S configuration, the 1gpu-16vcpu-200gb preset, Qwen/Qwen2.5-14B-Instruct, and vLLM inference.

Serverless Jobs support offline evaluation: running multiple synthetic scenarios, comparing detector predictions with ground truth, calculating precision, recall, F1 and latency, generating leaderboards, writing reports, and persisting artifacts. The representative Job configuration uses the cpu-d3 preset with four virtual CPUs and 16 GB of memory.

Using both products demonstrates more than a single inference request. The Endpoint handles interactive AI, while Jobs provide the repeatable batch-compute layer required by the benchmarking workflow.

Results

LOB Arena now demonstrates an end-to-end workflow from synthetic scenario generation to reviewable investigation and benchmark evidence.

Representative measurements were:

Workflow	Result
Local Docker startup and demo	approximately 3–5 minutes
Local ten-scenario tournament	approximately 0.7 seconds, excluding startup
Nebius Serverless Job with five scenarios	approximately 181 seconds
Serverless Job active compute cost	approximately \$0.005
Two Endpoint investigation requests	P50 24.2 seconds; P95 28.8 seconds
Endpoint request-active compute cost	approximately \$0.023

The estimated combined active compute cost of the representative Job and two Endpoint requests was approximately **\$0.028**.

These figures are workload-specific estimates based on measured active execution time. Actual billing can differ because of startup time, idle warm time, storage, billing increments, taxes, and pricing changes.

The detector metrics validate the synthetic evaluation and execution contracts. They should not be interpreted as claims about production-market surveillance accuracy.

What I learned

The most important design choice was separating “detect” from “explain.” In many AI prototypes, the model is asked to be both judge and storyteller. That is convenient, but difficult to evaluate.

In LOB Arena, detectors are deterministic functions over synthetic order-book state. They create incidents with explicit evidence. The AI layer translates that evidence into plain language, helps organize an investigation, narrates the result, or generates a bounded scenario. It does not silently replace detector logic or become the source of ground truth.

I also found that evidence transport is part of the product architecture. A successful cloud status is useful, but a reviewer needs the associated metrics, reports, logs, timestamps, and artifacts. Archiving to Object Storage, synchronizing through the backend, and exposing downloads in the UI turns an execution claim into an inspectable chain.

The next research steps are to increase benchmark and scenario diversity, develop adaptive adversarial agents, and measure detector degradation as market regimes change. I also want to calibrate synthetic behavior using publicly available market distributions, build richer incident replay tools, and compare deterministic and learned detectors under the same labels and evaluation contracts.

That comparison matters. A learned detector should not receive a more forgiving evaluation path because its internals are more complex. Deterministic and learned approaches should consume compatible evidence, preserve the same ground truth, report the same metric families, and produce artifacts another practitioner can inspect.

Public repository: <https://github.com/khab40/lob-arena>

Safety disclaimer: LOB Arena is synthetic and educational. It uses no real trading data, does not detect real market manipulation, does not provide trading signals, and is not suitable for compliance decisions.

#NebiusServerlessChallenge



Alexey Khabalov

Engineering Director / Senior Manager | Data Platforms, Market Data,
FinTech, AI | EU passport | Open to relocate globally within 1 month